

IoT Smart Home Dashboard - Interview Q&A Part 2

Core Features and Implementation Questions

USER AUTHENTICATION AND SECURITY

****Q13: How did you implement user authentication and what security measures did you include?****

****A:****

****Authentication Implementation**:**

```
```java
public boolean loginCustomer(String email, String password) {
 Customer customer = customerService.authenticateCustomer(email, password);
 if (customer != null) {
 sessionManager.login(customer);
 return true;
 }
 return false;
}
```
```

****Security Features**:**

1. ****Password Security**:**

- ****BCrypt Hashing**:** All passwords stored using BCrypt with salt
- ****Strength Requirements**:** 8-128 characters, uppercase, lowercase, numbers, special characters
- ****Common Password Prevention**:** Blocks weak passwords like "password123"

2. ****Account Protection**:**

- ****Failed Login Tracking**:** Monitors and increments failed attempts
- ****Account Lockout**:** Automatic lockout after multiple failures
- ****Session Management**:** Secure session handling with automatic cleanup

3. ****Input Validation**:**

- ****Email Validation**:** Regex pattern matching for valid email format
- ****SQL Injection Prevention**:** Parameterized queries through DynamoDB SDK
- ****XSS Prevention**:** Input sanitization for all user inputs

****Q14: Explain your password validation logic.****

****A:****

```
```java
public boolean isValidPassword(String password) {
 if (password == null || password.length() < 8 || password.length() > 128) {
 return false;
 }
}
```

```
// Check for required character types
boolean hasUpper = password.chars().anyMatch(Character::isUpperCase);
boolean hasLower = password.chars().anyMatch(Character::isLowerCase);
boolean hasDigit = password.chars().anyMatch(Character::isDigit);
boolean hasSpecial = password.chars().anyMatch(ch -> "!@#$%^&*".indexOf(ch) >= 0);

// Check for common passwords and repetition
return hasUpper && hasLower && hasDigit && hasSpecial
 && !isCommonPassword(password)
 && !hasExcessiveRepetition(password);
}
...

```

**\*\*Validation Rules\*\*:**

- **\*\*Length\*\***: 8-128 characters
- **\*\*Complexity\*\***: Must include uppercase, lowercase, digit, special character
- **\*\*Common Password Check\*\***: Prevents "password123", "qwerty", etc.
- **\*\*Repetition Check\*\***: Blocks passwords with >2 repeating characters

---

### ### DEVICE MANAGEMENT SYSTEM

**\*\*Q15: How does your device management system work?\*\***

**\*\*A:\*\***

**\*\*Device Architecture\*\*:**

```
```java
@DynamoDbBean
public class Gadget {
    private String type;      // Device category (TV, AC, FAN, etc.)
    private String model;     // Brand/model name
    private String roomName;  // Location identifier
    private String status;    // ON/OFF state
    private double powerRatingWatts; // Power consumption
    private LocalDateTime lastOnTime; // Session tracking
    private long totalUsageMinutes; // Cumulative usage
    private double totalEnergyConsumedKWh; // Energy consumption
}
...

```

****Key Features**:**

1. ****15+ Device Types Supported**:**

- ****Entertainment****: TV (150W), Speaker (30W)
- ****Climate Control****: AC (1500W), Fan (75W), Air Purifier (45W)
- ****Security****: Camera (15W), Door Lock (12W), Doorbell (8W)

- **Appliances**: Refrigerator (200W), Microwave (1200W), Geyser (2000W)

2. **Device Constraints**:

- **One device per room per type**: Prevents duplicate device registration
- **Room-based organization**: Logical grouping by location
- **Automatic power rating assignment**: Default watts based on device type

3. **Real-time State Management**:

- **Immediate status updates**: ON/OFF state changes instantly
- **Session tracking**: Records when devices are turned on/off
- **Energy calculation**: Real-time energy consumption updates

Q16: How do you handle device state changes and energy tracking?

A:

State Change Implementation:

```
```java
public void turnOn() {
 if (!isOn()) {
 this.lastOnTime = LocalDateTime.now();
 updateUsageAndEnergy(); // Calculate previous session energy
 }
 this.status = GadgetStatus.ON.name();
}

public void turnOff() {
 if (isOn()) {
 this.lastOffTime = LocalDateTime.now();
 updateUsageAndEnergy(); // Add current session to totals
 }
 this.status = GadgetStatus.OFF.name();
}
```
```

Energy Tracking Logic:

```
```java
private void updateUsageAndEnergy() {
 if (lastOnTime != null && isOn()) {
 long minutesUsed = ChronoUnit.MINUTES.between(lastOnTime, LocalDateTime.now());
 totalUsageMinutes += minutesUsed;

 double hoursUsed = minutesUsed / 60.0;
 double energyUsed = (powerRatingWatts / 1000.0) * hoursUsed;
 totalEnergyConsumedKWh += energyUsed;
 }
}
```
```

****Benefits**:**

- ****Accurate Energy Tracking****: Precise calculation based on actual usage time
- ****Session-based Recording****: Tracks individual usage sessions
- ****Real-time Updates****: Energy consumption updated on every state change

ENERGY MANAGEMENT SYSTEM

****Q17: Explain your energy management and cost calculation system.****

****A:****

****Energy Report Generation**:**

```
```java
public EnergyReport generateEnergyReport(Customer customer) {
 List<Gadget> devices = customer.getGadgets();
 double totalEnergyKWh = 0.0;

 // Calculate energy from all devices including current sessions
 for (Gadget device : devices) {
 totalEnergyKWh += device.getCurrentTotalEnergyConsumedKWh();
 }

 // Include deleted device energy for accurate monthly reporting
 totalEnergyKWh += customer.getTotalDeletedDeviceEnergyForCurrentMonth();

 double totalCost = calculateSlabBasedCost(totalEnergyKWh);
 return new EnergyReport(totalEnergyKWh, totalCost, reportPeriod, devices);
}
```
```

****Cost Calculation (Indian Electricity Tariff)**:**

```
```java
public double calculateSlabBasedCost(double totalKWh) {
 double totalCost = 0.0;

 if (totalKWh <= 30) {
 totalCost = totalKWh * 1.90; // â,1.90/kWh for first 30 units
 } else if (totalKWh <= 75) {
 totalCost = 30 * 1.90 + (totalKWh - 30) * 3.0; // â,13.00/kWh for next 45 units
 } else if (totalKWh <= 125) {
 totalCost = 30 * 1.90 + 45 * 3.0 + (totalKWh - 75) * 4.50; // â,14.50/kWh
 } else if (totalKWh <= 225) {
 totalCost = 30 * 1.90 + 45 * 3.0 + 50 * 4.50 + (totalKWh - 125) * 6.0;
 } else {
 // Above 225 units: â,16.60/kWh
 }
}
```
```

```

        totalCost = 30 * 1.90 + 45 * 3.0 + 50 * 4.50 + 100 * 6.0 + (totalKWh - 225) * 6.60;
    }

    return totalCost;
}
...

```

****Key Features**:**

- ****Real-time Energy Tracking****: Includes current session consumption
- ****Deleted Device History****: Maintains energy records even after device removal
- ****Tiered Pricing****: Implements realistic electricity slab rates
- ****Monthly Reporting****: Organized by billing cycles

****Q18: How do you handle deleted devices and maintain energy history?****

****A:****

****Deleted Device Energy Preservation**:**

```

```java
@DynamoDbBean
public class DeletedDeviceEnergyRecord {
 private String deviceType;
 private String roomName;
 private double totalEnergyConsumedKWh;
 private LocalDateTime deletionTime;
 private String deletionMonth; // "2024-12" format for monthly reporting
}
...

```

**\*\*Implementation\*\*:**

```

```java
public boolean deleteDevice(String deviceType, String roomName) {
    Gadget device = currentUser.findGadget(deviceType, roomName);
    if (device != null) {
        // Preserve energy data before deletion
        DeletedDeviceEnergyRecord record = new DeletedDeviceEnergyRecord(
            device.getType(),
            device.getRoomName(),
            device.getCurrentTotalEnergyConsumedKWh(),
            LocalDateTime.now()
        );
        currentUser.addDeletedDeviceRecord(record);

        // Remove device and update
        currentUser.getGadgets().remove(device);
        return customerService.updateCustomer(currentUser);
    }
    return false;
}

```

```
}  
...
```

****Benefits**:**

- ****Accurate Monthly Bills****: Energy consumption preserved even after device removal
- ****Historical Analysis****: Users can track energy patterns over time
- ****Data Integrity****: No energy data loss during device management

TIMER AND SCHEDULING SYSTEM

****Q19: How does your timer and scheduling system work?****

****A:****

****Timer Service Architecture**:**

```
```java  
public class TimerService {
 private final ScheduledExecutorService scheduler;

 private TimerService(CustomerService customerService) {
 this.scheduler = Executors.newScheduledThreadPool(5);
 startTimerMonitoring(); // Background monitoring
 }

 private void startTimerMonitoring() {
 scheduler.scheduleAtFixedRate(this::checkAndExecuteDueTimers, 0, 30, TimeUnit.SECONDS);
 }
}
...`
```

**\*\*Timer Task Structure\*\*:**

```
```java  
public static class TimerTask {  
    private final String deviceType;  
    private final String roomName;  
    private final String action; // "ON" or "OFF"  
    private final LocalDateTime scheduledTime;  
    private final String userEmail;  
}  
...`
```

****Scheduling Implementation**:**

```
```java  
public boolean scheduleDeviceTimer(String deviceType, String roomName, String action, String
dateTimeStr) {
 try {
```

```

 LocalDateTime scheduledTime = parseDateTime(dateTimeStr); // "DD-MM-YYYY HH:MM"

 if (scheduledTime.isBefore(LocalDateTime.now())) {
 System.out.println("Cannot schedule timer in the past!");
 return false;
 }

 // Update device with scheduled time
 Gadget device = currentUser.findGadget(deviceType, roomName);
 if ("ON".equals(action)) {
 device.setScheduledOnTime(scheduledTime);
 } else {
 device.setScheduledOffTime(scheduledTime);
 }

 return customerService.updateCustomer(currentUser);
 } catch (Exception e) {
 return false;
 }
}
...

```

**\*\*Key Features\*\*:**

- **\*\*Background Monitoring\*\***: 30-second interval timer checks
- **\*\*Flexible Scheduling\*\***: Support for both ON and OFF timers
- **\*\*Date Format Support\*\***: User-friendly "DD-MM-YYYY HH:MM" format
- **\*\*Past Date Prevention\*\***: Validates future scheduling only

**\*\*Q20: How do you handle concurrent timer execution?\*\***

**\*\*A:\*\***

**\*\*Thread Pool Management\*\*:**

```

```java
private final ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(5);
...

```

****Concurrent Execution Handling**:**

```

```java
private void checkAndExecuteDueTimers() {
 try {
 List<Customer> allCustomers = customerService.getAllCustomers();
 LocalDateTime now = LocalDateTime.now();

 for (Customer customer : allCustomers) {
 processCustomerTimers(customer, now);
 }
 } catch (Exception e) {

```

```

 System.err.println("Error in timer monitoring: " + e.getMessage());
 }
}
...

```

#### **\*\*Safety Measures\*\*:**

- **\*\*Thread Pool\*\***: Limited to 5 concurrent threads to prevent resource exhaustion
- **\*\*Exception Isolation\*\***: Individual timer failures don't affect others
- **\*\*Database Consistency\*\***: Each timer execution is atomic
- **\*\*Retry Logic\*\***: Failed timer executions are logged and can be retried

---

### **### ALERT SYSTEM**

**\*\*Q21: Explain your alert system implementation.\*\***

**\*\*A:\*\***

#### **\*\*Alert Types\*\*:**

```

```java
public enum AlertType {
    TIME_BASED,    // Trigger at specific date/time
    ENERGY_USAGE  // Trigger based on energy consumption
}
...

```

****Alert Structure**:**

```

```java
public static class Alert {
 private String alertId;
 private String alertName;
 private String deviceType;
 private String roomName;
 private AlertType alertType;
 private String condition; // "GREATER_THAN:10.5" or "2024-12-25T18:30"
 private String message;
 private boolean isActive;
 private LocalDateTime createdTime;
 private LocalDateTime lastTriggered;
 private int triggerCount;
 private boolean autoDeleteAfterTrigger;
}
...

```

#### **\*\*Time-Based Alert Creation\*\*:**

```

```java
public boolean createTimeBasedAlert(String alertName, String deviceType, String roomName,

```



```

        LocalDateTime triggerTime, String message) {
String alertId = generateAlertId();
String condition = triggerTime.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME);

Alert alert = new Alert(alertId, alertName, deviceType, roomName,
        AlertType.TIME_BASED, condition, message);

return addAlert(currentUserEmail, alert);
}
...

**Energy Usage Alert Creation**:
``java
public boolean createEnergyUsageAlert(String alertName, String deviceType, String roomName,
        double threshold, String comparisonType, String message) {
String condition = comparisonType + ":" + threshold; // "GREATER_THAN:10.5"

Alert alert = new Alert(alertId, alertName, deviceType, roomName,
        AlertType.ENERGY_USAGE, condition, message);

return addAlert(currentUserEmail, alert);
}
...

**Alert Processing**:
``java
public void checkAndTriggerAlerts(String userEmail, List<Gadget> devices) {
List<Alert> userAlerts = getUserAlerts(userEmail);
LocalDateTime now = LocalDateTime.now();

for (Alert alert : userAlerts) {
    if (!alert.isActive()) continue;

    boolean shouldTrigger = false;

    if (alert.getAlertType() == AlertType.TIME_BASED) {
        LocalDateTime triggerTime = LocalDateTime.parse(alert.getCondition());
        shouldTrigger = now.isAfter(triggerTime) &&
            (alert.getLastTriggered() == null ||
                !isSameMinute(alert.getLastTriggered(), triggerTime));
    } else if (alert.getAlertType() == AlertType.ENERGY_USAGE) {
        shouldTrigger = evaluateEnergyCondition(alert, devices);
    }

    if (shouldTrigger) {
        triggerAlert(alert);
    }
}
}

```

```
}  
...
```

****Features**:**

- ****Dual Alert Types****: Time-based and energy-threshold alerts
- ****Flexible Conditions****: Support for >, <, = comparisons
- ****Auto-deletion****: Optional automatic removal after trigger
- ****Trigger Tracking****: Prevents duplicate notifications
- ****Device-specific****: Alerts tied to specific devices

This concludes Part 2 covering Core Features and Implementation details.

****Next****: Part 3 will cover Advanced Features and System Design questions.